IBM i Source Repository — Git Project Split Plan

Date: 2026-03-06 **Prepared by:** LONGDM **Source Directory:** `production_source/ilesrc` **Total Files:** ~23,920

Executive Summary

This document proposes splitting the monolithic IBM i source repository (~23,920 files) into **16 logical git projects** organized by business function. The guiding principles:

1. **All database definitions (PF, LF, SQL DDL) go into a single dedicated project**
2. **Display files stay with their related programs** in functional projects
3. **Projects are organized by business domain** (billing, credit card, member, etc.)
4. **Shared service programs get their own project** as a common dependency

---

Current Inventory

File Type	Count	Description
`.lf`	4,796	Logical Files
`.sql`	4,063	SQL DDL (tables, indexes, views, triggers)
`.rpgle`	3,815	RPG IV programs
`.pf`	3,151	Physical Files
`.clp`	2,642	CL Programs (classic)
`.sqlrpgle`	1,899	RPG IV with embedded SQL
`.dspf`	1,616	Display Files
`.clle`	599	CL Programs (ILE)
`.txt`	544	Text/documentation
`.lf38`	224	Legacy Logical Files
`.prtf`	218	Printer Files
`.pf38`	114	Legacy Physical Files
`.rpg`	59	Legacy RPG III
`.bnd`	29	Binder Source
`.cmd`	30	Command Definitions
`.rpgleinc`	10	RPG Include Files
Other	~51	srcmbr, icff, pnlgrp, etc.

Proposed Project Structure

Dependency Architecture

```
                    ibmi-database
                         |
                  ibmi-shared-services
                 /    |    |    \     \
                /     |    |     \     \
   ibmi-srq  ibmi-tvc  ...  ...   ...  ibmi-misc
```

Every functional project depends on `ibmi-database` and `ibmi-shared-services`. No functional project should depend on another functional project.

Project Definitions

1. `ibmi-database` — Database Object Definitions

Purpose: Single source of truth for all physical files, logical files, and SQL DDL.

Content	Count
`.pf` Physical Files	3,151
`.pf38` Legacy Physical Files	114
`.lf` Logical Files	4,796
`.lf38` Legacy Logical Files	224
`.sql` DDL (tables, indexes, views, triggers)	~4,063

Estimated total: ~12,200 files (51% of repository)

Internal organization:

```
ibmi-database/
  pf/            — Physical files (subdirs by business area)
  lf/            — Logical files (subdirs by business area)
  sql/
    tables/
    indexes/
    views/
    triggers/
    procedures/
    functions/
  legacy/        — .pf38, .lf38 files
```

Rationale: Centralizes all schema definitions. Enables schema change tracking, dependency analysis, and consistent DDL review. Enforces clear compile order (PFs → LFs → SQL indexes).

---

2. `ibmi-shared-services` — Service Programs, Headers, and Utilities

Purpose: Shared service programs used across multiple business domains.

Contents:

- All 29 `.bnd` binder source files
- All 10 `.rpgleinc` include files
- All 38 `*_h.*` header files
- Service program implementations (`.sqlrpgle` / `.rpgle`)
- All 30 `.cmd` command definitions
- User management files (`usrlogin*` , `usrrole*` , `userc*`)

Service programs included:

Service	Description
`addrsvc`	Address service
`cache`	Caching
`crcdb`	Credit card database access
`dscnt`	Discount service
`entl`	Entitlement service
`gb`	Group benefits
`httpsvc`	HTTP service wrapper
`idnt`	Identity service
`ldarkly`	LaunchDarkly feature flags
`log`	Logging
`misc`	Miscellaneous utilities
`offers` / `offrmap` / `ofrsync`	Offers management
`pfclb`	Prepaid Legal club
`pmts` / `pmtsvc`	Payment services
`prof` / `profinq`	Profile service
`queries`	Query utilities
`subs`	Subscription service
`sync` / `sync2` / `sync3` / `sync4`	Synchronization services
`taxcalc`	Tax calculation
`test`	Test utilities
`tokenx`	Token exchange
`user`	User management
`vertex`	Vertex tax integration

Estimated total: ~280 files

---

3. `ibmi-srq` — Service Request / Queue System

Purpose: The largest functional module — the SRQ workflow/queue management system.

File patterns: `srq*` (all types except PF/LF/SQL-DDL)

Estimated total: ~2,260 files (programs, display files, printer files)

Rationale: At 1,761+ files by prefix, SRQ is by far the largest subsystem and operates as a self-contained request processing engine.

---

4. `ibmi-tvc` — Time Value Compensation / Associate Commission

Purpose: Associate compensation system including TVC base, reports, control, agents, and events.

File patterns: `tvc*`, `tvcr*`, `tvcc*`, `tvccl*`, `tssnr*`, `tmnor*`

Estimated total: ~1,650 files

Rationale: Second-largest subsystem. TVC handles compensation hierarchy and calculations. `tssnr` (time/salary) and `tmnor` (time reports) feed directly into compensation.

---

5. `ibmi-billing` — Billing and Financial Processing

Purpose: Billing file processing, billing reports, cash worksheets.

File patterns: `bilfile*`, `bilfil*`, `bill*`, `bilr*`, `cashwk*`, `cash*`, `kpibilf*`, `msbilf*`, `acctcash*`

Estimated total: ~550 files

---

6. `ibmi-credit-card` — Credit Card Processing

Purpose: Credit card billing, authorization, renewals, and payment card processing.

File patterns: `crcd*`, `ccahist*`, `ccauth*`, `ccrand*`, `ccrenew*`, `vgs*`

Estimated total: ~420 files

Rationale: Security-sensitive domain. Isolation supports PCI compliance review.

---

7. `ibmi-member` — Member Management

Purpose: Member master maintenance, documents, enrollment, dependent management.

File patterns: `mbr*`, `mbrc*`, `mbrd*`, `mbri*`, `mbrr*`, `meclnf*`, `mbelife*`, `dep*`, `dp*`, `olmbr*`, `mbrdoc*`

Estimated total: ~450 files

---

8. `ibmi-group` — Group Management

Purpose: Group master maintenance, group benefits workbench, group data and reporting.

File patterns: `grp*`, `grpc*`, `grpr*`, `gbwb*`, `gbwr*`, `grmr*`, `gdar*`

Estimated total: ~640 files

---

9. `ibmi-payments` — Payment Processing

Purpose: Payment management, banking/ACH processing, payment reports.

File patterns: `pym*`, `pmt*`, `pml*`, `paybatch*`, `bnk*`, `bok*`, `bk1*`, `bk2*`, `ppr*`, `pcheck*`

Also includes: `.icff` files (8) — banking communication definitions

Estimated total: ~420 files

---

10. `ibmi-commission` — Commission and Compensation

Purpose: Commission calculations, statements, referral compensation.

File patterns: `co[0-9]*`, `comm*`, `adjcom*`, `refc*`, `refd*`, `refr*`

Estimated total: ~230 files

Note: Distinct from TVC — these handle dollar-level calculations and statements, while TVC handles the compensation hierarchy.

---

11. `ibmi-application-entry` — Application Entry and Enrollment

Purpose: New membership applications, enrollment, 834 EDI processing.

File patterns: `aer*`, `aeapps*`, `aecl*`, `aepend*`, `ahapps*`, `ahassoc*`, `apee*`, `frapps*`, `genweb*`, `websign*`

Estimated total: ~350 files

---

12. `ibmi-plans-benefits` — Plans, Benefits, and Entitlements

Purpose: Plan definitions, plan comparison, benefits generation, provider network.

File patterns: `plan*`, `plcr*`, `plcc*`, `bgen*`, `egen*`, `eb*`, `erisa*`, `pfs*`, `conserv*`, `prov*`

Estimated total: ~480 files

---

13. `ibmi-claims-letters` — Claims Processing and Correspondence

Purpose: Claims management, letter generation, cancellation, reinstatement, returns.

File patterns: `clm*`, `clmr*`, `clmc*`, `ltr*`, `lett*`, `cnl*`, `can*`, `cancel*`, `precan*`, `reinst*`, `return*`, `rtn*`, `cert*`, `uncp*`

Estimated total: ~470 files

Note: Contains the bulk of printer files — 40+ letter templates, cancellation notices, reinstatement letters, etc.

---

14. `ibmi-reporting-kpi` — Reporting, KPI, and Analytics

Purpose: Cross-functional reports, KPI dashboards, comparison tools.

File patterns: `kpi*`, `rpt*`, `cmpr*`, `cmp*`, `rcr*`, `wsar*`, `drbal*`, `cshr*`

Estimated total: ~680 files

Rationale: Reporting spans multiple domains. Grouping reports together prevents every project from having a long reporting tail.

---

15. `ibmi-web-integration` — Web Services, APIs, and Integration

Purpose: Web-facing services, Atlas/Adonis sync, external integrations.

File patterns: `web*` (remaining after `websign*`), `atlas*`, `adon*`, sync process runners (`syncprc*`), `img*`, `svy*`

Estimated total: ~250 files

Note: Sync service program implementations stay in `ibmi-shared-services`. Sync process orchestration files live here.

---

16. `ibmi-misc` — Miscellaneous, Legacy, and Uncategorized

Purpose: Catch-all for files without clear domain ownership.

Contents include:

- Legacy RPG III programs (`p[0-9]*`, etc.)
- Single-letter prefix batch jobs (`j[0-9]*`)
- Tax/1099 processing (~101 files)
- Small prefix groups with unclear ownership
- Employee time files (`etr*`, `etcl*`) — future candidate for own project
- Calendar/call processing (`cal*`)
- Account maintenance (`acm*`, `acd*`)
- Sales orders (`sooe*`, `soord*`, `soitm*`)
- Marketing (`mkmg*`)
- Business channel (`bus*`)

Estimated total: ~3,300 files

Strategy: Progressively empty this project by identifying business ownership and moving files to appropriate domain projects over time.

Printer File (.prtf) Distribution

Project	Printer Files	Examples
`ibmi-claims-letters`	~60	`ltr*`, `cancel*`, `reinst*`, `return*`, `cert*`
`ibmi-group`	~15	`grp*`, `gbwstmt*`, `delgrp*`
`ibmi-srq`	~11	`srq00023p`, `srq00042p`
`ibmi-billing`	~10	`bill050p`, `dirstmt*`
`ibmi-tvc`	~9	`tvc*`, `tvcr*`
`ibmi-reporting-kpi`	~10	`kpi*`, `rcpr*`, `mas*`
`ibmi-payments`	~8	`pmt*`, `ppr*`
`ibmi-commission`	~6	`co*`, `comm*`
`ibmi-misc`	~89	Everything else (incl. tax/1099 forms)

File Count Summary

#	Project	Est. Files	%
1	`ibmi-database`	~12,200	51%
2	`ibmi-shared-services`	~280	1%
3	`ibmi-srq`	~2,260	9%
4	`ibmi-tvc`	~1,650	7%
5	`ibmi-billing`	~550	2%
6	`ibmi-credit-card`	~420	2%
7	`ibmi-member`	~450	2%
8	`ibmi-group`	~640	3%
9	`ibmi-payments`	~420	2%
10	`ibmi-commission`	~230	1%
11	`ibmi-application-entry`	~350	1.5%
12	`ibmi-plans-benefits`	~480	2%
13	`ibmi-claims-letters`	~470	2%
14	`ibmi-reporting-kpi`	~680	3%
15	`ibmi-web-integration`	~250	1%
16	`ibmi-misc`	~3,300	14%
	Total	**~24,630**	

> *Note: Total exceeds 23,920 due to prefix overlap in estimates. During actual migration, each file goes to exactly one project.*

Migration Strategy

Phase 1: Database Project (Week 1-2)

1. Extract ALL `.pf`, `.pf38`, `.lf`, `.lf38` files — these are unambiguous
2. Extract `.sql` files containing DDL (CREATE TABLE, INDEX, VIEW, triggers)
3. Validate: confirm program compilations still work with same library list
4. **Impact:** Moves ~12,200 files (51% of the repo) in one step

Phase 2: Shared Services (Week 2)

1. Move all `.bnd`, `.rpgleinc`, `*_h.*` files
2. Move service program implementation files
3. Move all `.cmd` files
4. **Impact:** ~280 files, clean extraction

Phase 3: The Big Two (Week 3-4)

1. Extract `ibmi-srq` — straightforward `srq*` prefix extraction
2. Extract `ibmi-tvc` — `tvc*`, `tssnr*`, `tmnor*` prefix extraction

3. **Impact:** ~3,900 files

Phase 4: Remaining Functional Projects (Week 5-8)

1. Work through each domain project in order of size
2. For each: extract by prefix, verify no orphaned references, update build scripts
3. Handle ambiguous files case-by-case

Phase 5: Triage ibmi-misc (Ongoing)

1. Review remaining files quarterly
2. Assign business ownership to orphan prefix groups
3. Move files to appropriate projects as ownership is determined
4. **Target:** Reduce `ibmi-misc` to <500 files within 6 months

Build System Considerations

- Each project needs its own build configuration referencing `ibmi-database` and `ibmi-shared-services` as dependencies
- Library lists on IBM i must include database and shared-services objects
- CI/CD pipelines should build in dependency order: database → shared-services → functional projects (parallelizable)

Git History Preservation

Option A (Recommended): Keep the original monorepo as a read-only archive. Start fresh history in each new project. Simple and clean.

Option B: Use `git filter-repo` to preserve file history during the split. More complex but retains blame/log history per file.

Open Questions for Team Discussion

1. Should `etr*` / `etcl*` (employee time, ~108 files) stay in `ibmi-misc` or move to `ibmi-tvc`?
2. Should tax/1099 processing (~101 files) get its own project `ibmi-tax` or remain in `ibmi-misc`?
3. Should sales order files (`sooe*` , `soord*` , ~177 files) get their own project?
4. Are there cross-domain program calls that would need to be refactored into shared services?
5. What is the preferred CI/CD tooling for building across multiple git repos?